International Institute of Information Technology Bangalore

upGrad

Introduction to Neural Networks

Tulika Saha

International Institute of Information Technology Bangalore, India

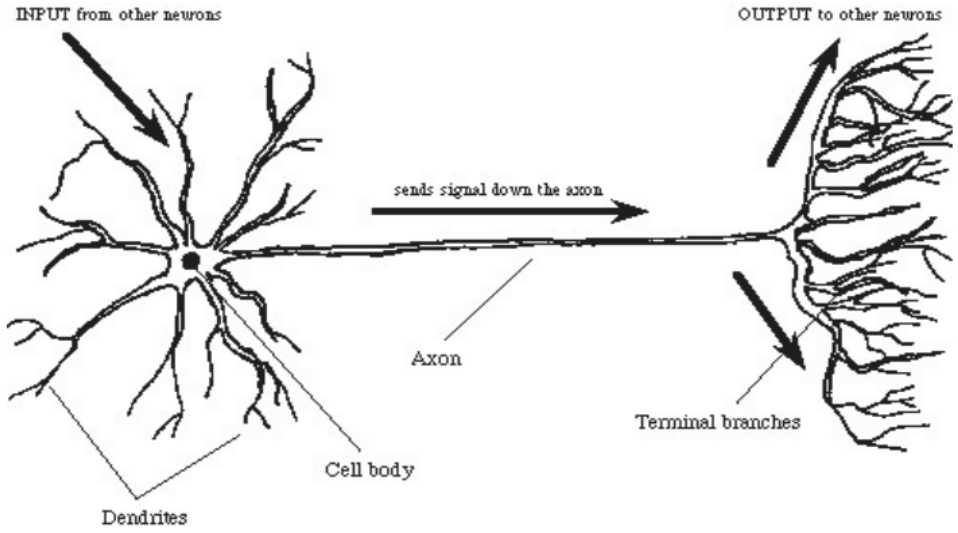
Agenda

- Introduction
- Perceptron
- Artificial Neural Networks
- Backpropagation Algorithm
- Demonstration

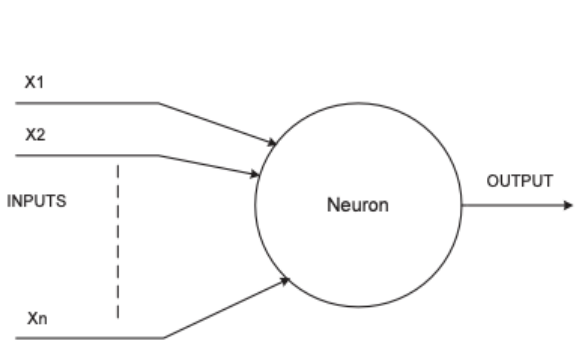
Section 1: Introduction

Brains & (A)NNs

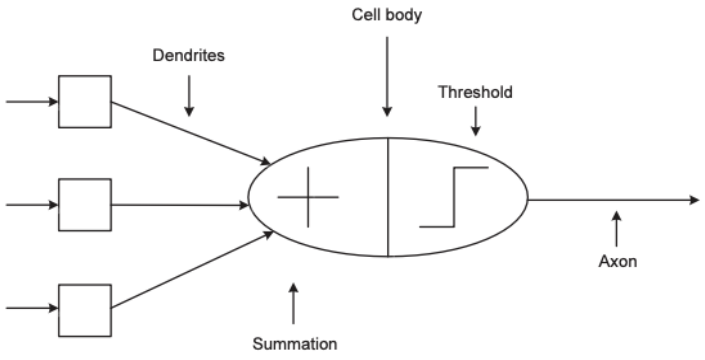
- Artificial Neural Networks (or Neural Networks) have been motivated by the recognition that the *human brain* computes in a way vastly different from a *conventional computer*
- A brain is a *highly complex*, *nonlinear*, massively parallel **computing machine** (i.e., supports information processing), which uses simple internal structural constituents, the **neurons**, to facilitate and support motor control, perception, pattern recognition, etc.
- **ANNs are biologically inspired analogues of the brain**, and are also based on primitive neuron components. Both brains and ANNs acquire and store information supplied by the environment through a **learning process**, and represent knowledge using **inter-neuron connection strengths** (synaptic weights).



Neuron structure in a brain



Neuron structure in a NN

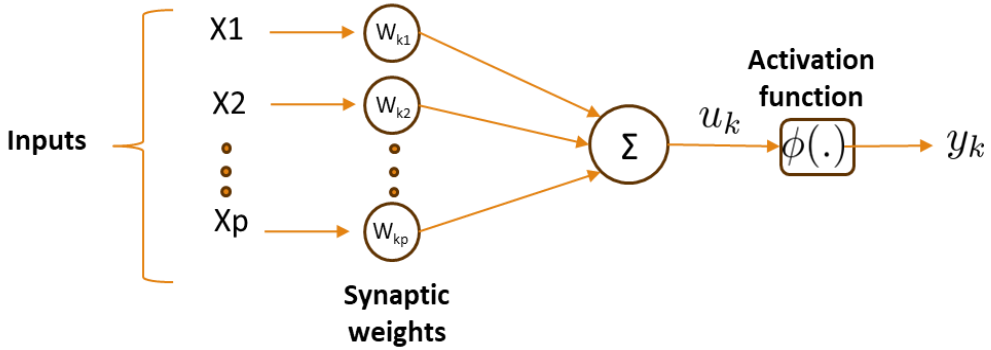


Detailed neuron structure in a NN

Section 2: Perceptron

Structural Aspects of a Neuron

1. Each neuron is an information processing unit fundamental to the entire operation of the ANN. The basic elements of a typical neuron are:
   a) A set of **synapses or connections**. Each of these is characterised by a weight (strength). The j^{th} synapse connected to the k^{th} neuron receives signal x_j and multiplies it by w_{kj}.
   b) An **adder** for summing the input signals, weighted by the respective synapses. The combined synaptic signals are denoted by u_k.
   c) An **activation function** ϕ, which squashes the permissible amplitude range of the output signal. The final output is denoted by y_k.



The mathematical representation of the above diagram:

$$u_k = \sum_{j=1}^{p} w_{kj} x_j = w_k^T x, \qquad y_k = \phi(u_k)$$

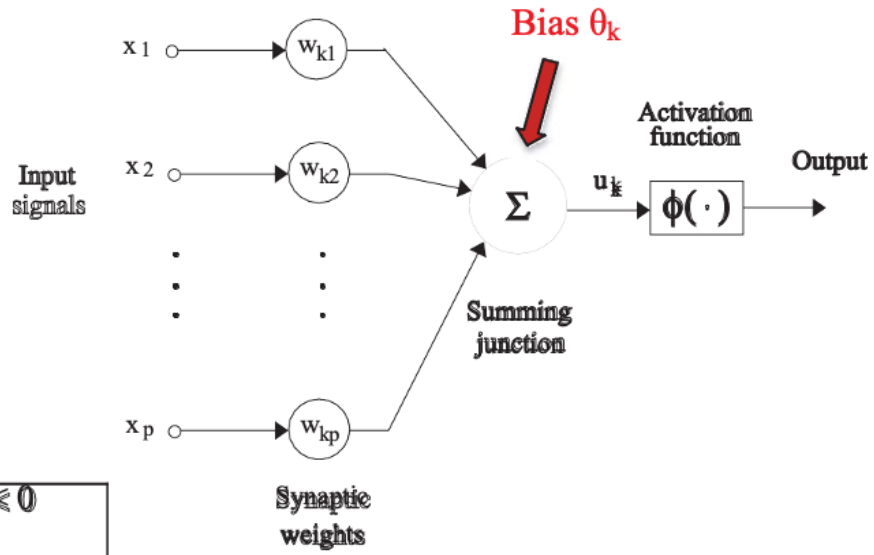
Structural Aspects of a Neuron

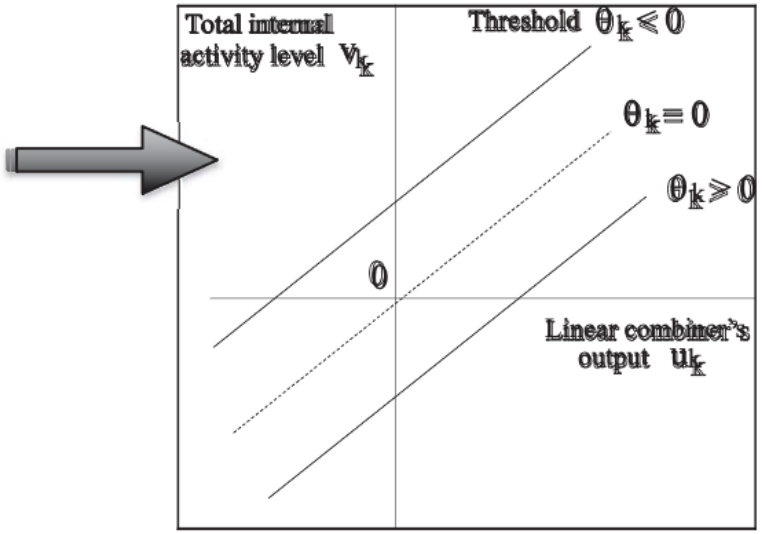
- The externally applied bias θ_k decreases or increases the internal input to the activation
- Thus, the output becomes:

$$y_k = \phi\left(\mathbf{w}_k^T \mathbf{x} - \theta_k\right)$$

where, $v_k = u_k - \theta_k$

- Overall, the bias applies an affine transformation to the output u_k and modifies the **induced local field** or activation potential v_k.





Structural Aspects of a Neuron

- For conciseness, the bias can be incorporated as a weight $w_{k0} = \theta_k$

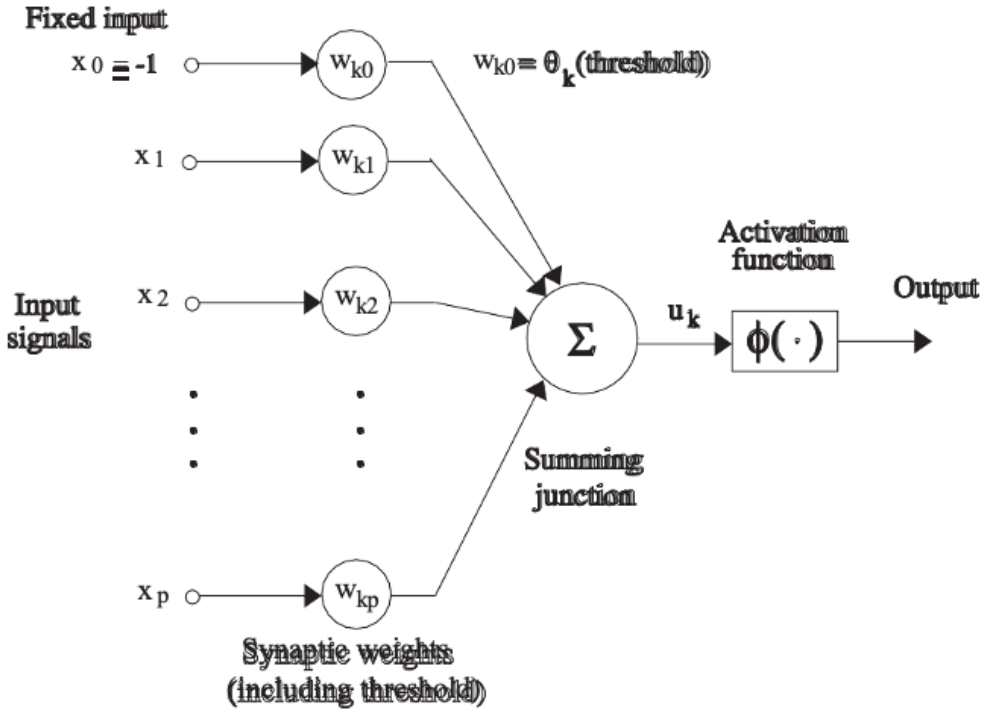
via a fixed input $x_O = -1$. Thus, the model becomes:

$$u_k = \sum_{j=0}^{p} w_{kj} x_j$$

$$y_k = \phi(u_k)$$

$$\text{where} \quad \mathbf{x} = (-1, x_1, \ldots, x_p)$$

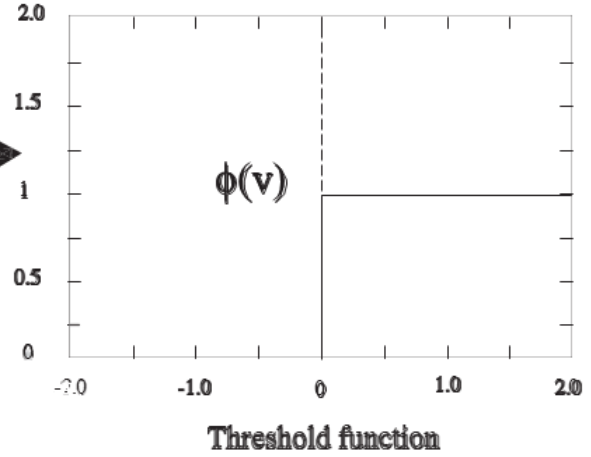
$$\text{and} \quad \mathbf{w}_k = (\theta_k, w_{k1}, \ldots, w_{kp})$$



Types of Activation Function

- The activation function ϕ(v), where v is the induced local field, defines the actual neuron output. There are various ways this can be implemented:
- **Threshold or Heaviside function.** This has a binary output:

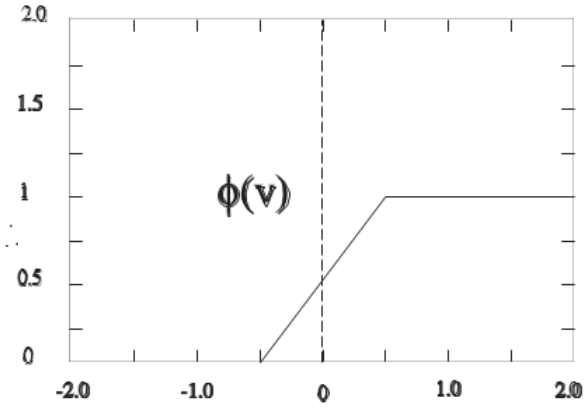
$$\phi(v) = \begin{cases} 1 & \text{if} \quad v \geq 0 \\ 0 & \text{if} \quad v < 0 \end{cases}$$



Threshold function

- **Piecewise linear function.** This corresponds to a nonlinear (linear truncated) amplification.

$$\phi(v) = \begin{cases} 1 & \text{if} \quad v \geq +\tfrac{1}{2} \\ v + \tfrac{1}{2} & \text{if} \quad -\tfrac{1}{2} < v < +\tfrac{1}{2} \\ 0 & \text{if} \quad v \leq -\tfrac{1}{2} \end{cases}$$

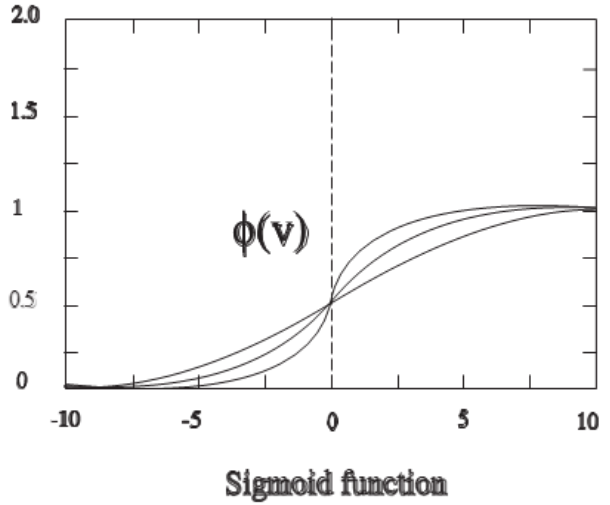


Piecewise-linear function

Types of Activation Function

- **Sigmoid function.** This produces a differentiable S-shaped nonlinear amplification of the field. Possible expressions are shown below. α is the slope of the Logistic sigmoid (when α-->∞ it becomes a Heaviside function). This function is continuous and ranges within (0,1).

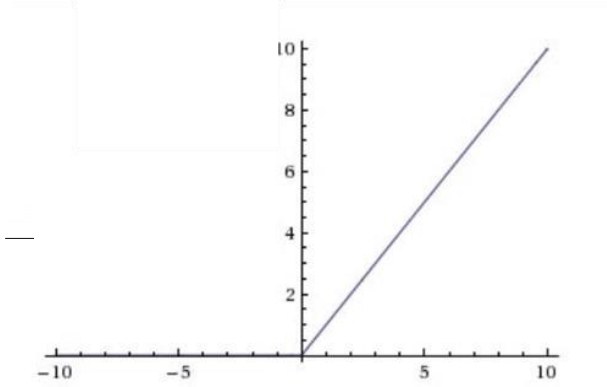
$$\phi(v) = \frac{1}{1+\exp(-\alpha \cdot v)} \quad \text{or} \quad \phi(v) = \tanh(v) = \frac{\exp(2v)-1}{\exp(2v)+1} \in (-1,+1)$$



Sigmoid function

- **Rectified linear unit (RELU).** Most common choice in state-of- the-art neural networks & deep learning:

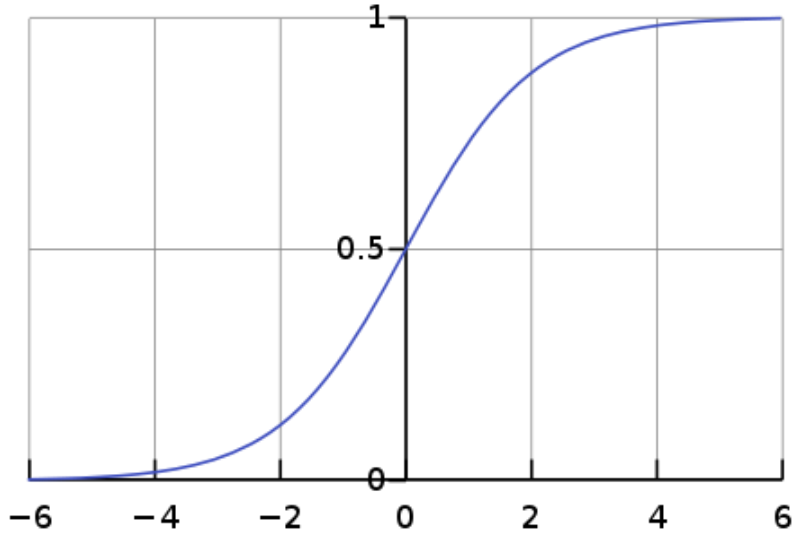
$$\phi(v) = \max(v, 0) = \begin{cases} v & v \geq 0 \\ 0 & v < 0 \end{cases}$$



Types of Activation Function

- **Softmax.** This is a mathematical function which takes a vector of K real numbers as input and converts it into a probability distribution of K probabilities proportional to the exponential of input numbers.

$$\phi(v_i) = \frac{e^{v_i}}{\sum_{j=1}^{K} e^{v_j}}$$



Error-Correction Learning

- Assume that the k^{th} neuron receives a signal vector **x**(n), where n is the time variable of a discrete synaptic adaptation procedure. This signal can be the result of other neurons in previous layers.
- The neuron **output** signal is denoted by $y_k(n)$.
- The **desired** output signal is denoted by $d_k(n)$.
- Therefore, we can define the **error** signal: $e_k(n) = d_k(n) - y_k(n)$ => $\mathbf{e} = \mathbf{d} - X.\mathbf{w}$
- This constitutes a control mechanism by which a sequence of corrective step-by-step adjustments are driven to bring the output y_k closer to the desired d_k.
- This is achieved by minimising a cost function E(n) until the system reaches a steady-state (synaptic weights are stabilised):

$$E(n) = \frac{1}{2}e_k^2(n)$$

=>

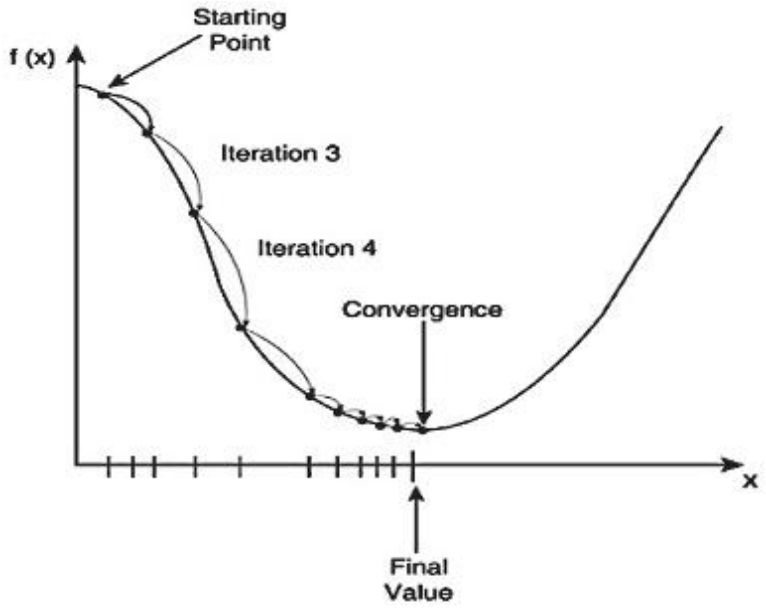
$$\frac{dE}{dW} = \frac{\partial E}{\partial e} \times \frac{\partial e}{\partial W}$$

$$\frac{dE}{dW} = \frac{2}{2}e(0 - x) = -ex$$

$$\frac{dE}{dw_{kj}} = -e_k.x_j$$

Error-Correction Learning

- Employing a positive learning parameter η, this corresponds to the **Delta or Widrow-Hoff rule**:



$$\Delta w_{kj}(n) = \eta e_k(n) x_j(n)$$

$$\underbrace{w_{kj}(n+1)}_{\text{new weight}} = \underbrace{w_{kj}(n)}_{\text{old weight}} + \Delta w_{kj}(n)$$

- With this rule, the synaptic weight adjustment is **proportional** to the error signal and the input signal of the synapse under adjustment.

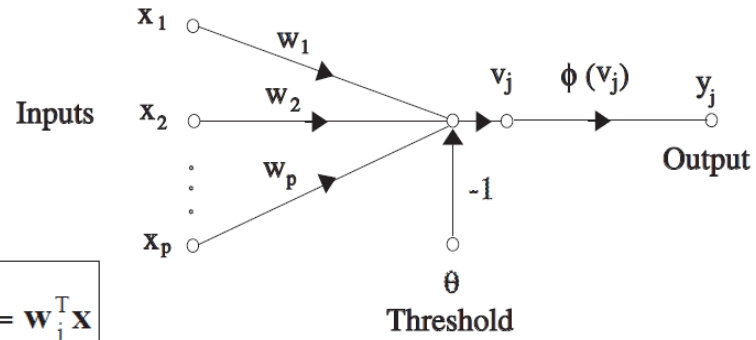
Perceptrons

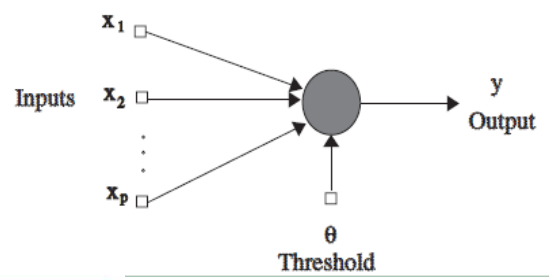
- Rosenblatt's Perceptrons are based on the McCulloch-Pitts neuronal model.
  They are similar to LMS, but they have nonlinear activations based on thresholding activati

$$v_j = \sum_{i=1}^{p} w_{ji} x_i - \theta_j = \mathbf{w}_j^T \mathbf{x}$$

$$y_j = \phi(v_j) \in \{-1, +1\}$$

$$\text{e.g., } \phi(v_j) = \text{signum}(v_j) = \begin{cases} +1 & \text{if } v_j > 0 \\ -1 & \text{if } v_j < 0 \end{cases}$$

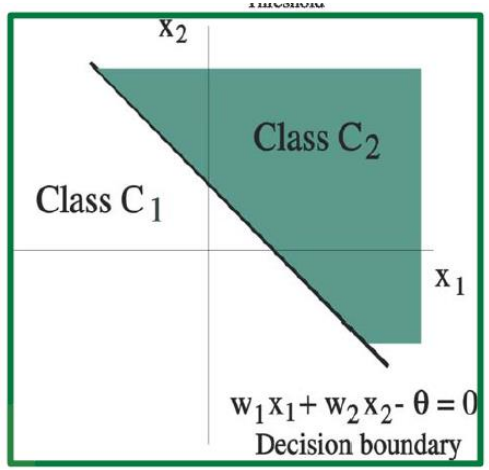




- The goal of the Perceptron is to correctly classify a set of external stimuli to one of two given classes C1 or C2.
  - This means that the internal representation is a hyperplane in the p-dimensional space which separates the space into two class coding regions.
  - This hyperplane is the **decision boundary** (defined as of the system's intelligence.

$\sum_{i=1}^{p} w_i x_i = \theta)$

2D example



Perceptron Training

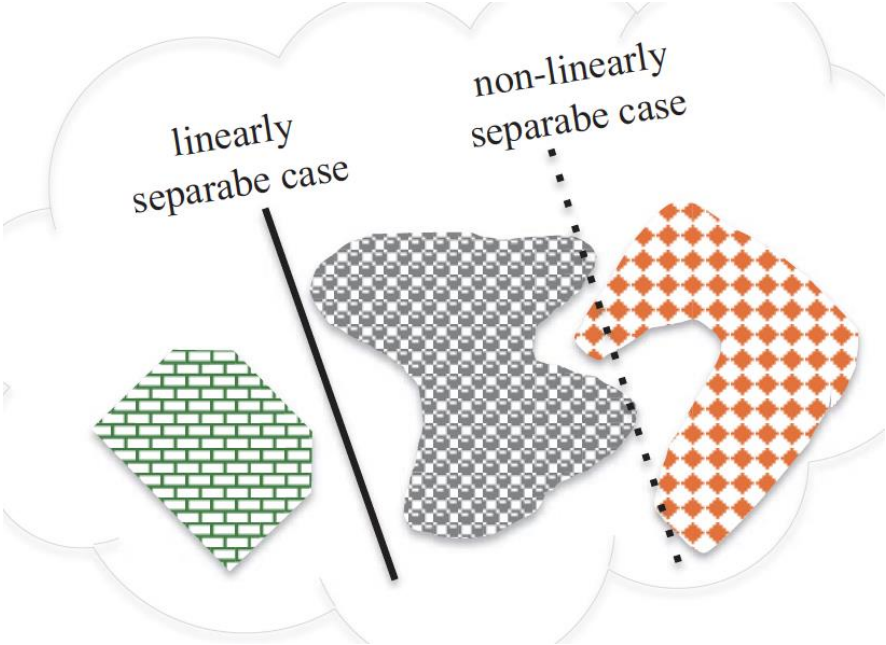
- An error-correction learning algorithm is used to train the perceptron (to adjust weights, or position & rotate the decision boundary).
- Since the decision boundary is linear, only linearly separable patterns can be fully classified by the perceptron correctly.

• Assume that the dataset S_{train} is split into two subsets, D_1 and D_2 containing the samples from classes C_1 and C_2, respectively.

•Then if we can find some vector **w** that satisfies both of the following inequalities, the samples in D are linearly separable.

$$\left\{ \begin{array}{ll} \mathbf{w}^T\mathbf{x} > 0 & \forall \mathbf{x} \in D_1 \\ \mathbf{w}^T\mathbf{x} \leq 0 & \forall \mathbf{x} \in D_2 \end{array} \right\} \quad \text{where: } D_1 \cap D_2 = \varnothing, D_1 \cup D_2 = D_{train}$$

•The problem of training the Perceptron, now simply becomes a problem of finding one possible weight vector **w**, which satisfies the above inequalities.



Perceptron Training Algorithm

- Step 1 (Initialisation): Set t=1, $\mathbf{w}(1)=\mathbf{0}$, and learning rate η in (0,1]. (*Note: small* η yields averaging of past inputs and stable estimates, while large η yields fast learning and fast adjustment to environmental changes).
- Step 2 (Activation): Apply the sample input $\mathbf{x}(t)$ to the neuron.
- Step 3 (Response): Compute its response:

$$y(t) = \text{signum}\left[\mathbf{w}(t)^T \mathbf{x}(t)\right]$$

- Step 4 (Adaptation): Update the current weight vector via:

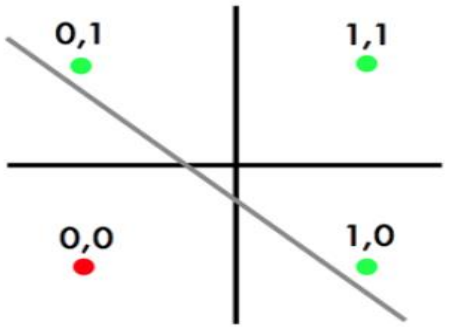
$$\mathbf{w}(t+1) = \mathbf{w}(t) + \eta\underbrace{\left[d(t) - y(t)\right]}_{\text{error signal} \in \{0,\pm 2\}}\mathbf{x}(t)$$

$$d(t) = \begin{cases} +1 & \text{if } \mathbf{x}(t) \text{ belongs to } C_1 \\ -1 & \text{if } \mathbf{x}(t) \text{ belongs to } C_2 \end{cases}$$

- Step 5 (Continuation): Unless all samples are classified correctly, set t=t+1 and go to step 2.

A classification problem using SLP

- Assume you have a perceptron with two inputs denoted by x_1 and x_2, and one threshold corresponding to a fixed input denoted by x_0=-1. The perceptron is based on a Signum activation function of output values +1 (for non-negative input only) and -1.
- Starting with the initial weights w_1=+1.0, w_2=+1.0, and an initial threshold w_0=-1.5, calculate all the steps and weight adjustments involved with its training to learn a two-input OR problem.
- To achieve this, use the Perceptron training algorithm with a learning rate η=0.5.
- During training, in each epoch, present the input patterns in the following order: (0,0), (0,1), (1,0) and (1,1). Also, map the "false" (or 0) output to class label -1 and the "true" to +1.



OR

Solution

- First write the Perceptron model for its output:

$$y = \phi\left[\mathbf{w}^T\mathbf{x}\right] = \phi\left[\sum_{i=0}^{2} x_i w_i\right], \quad \text{where} \quad x_0 = -1 \text{ (fixed)} \qquad \phi[v] = \begin{cases} +1 & v \geq 0 \\ -1 & v < 0 \end{cases}$$

- Then, we write down the complete patterns for the OR problem, and the desired response d (converted to -1 and +1):

Pattern	x_0	x_1	x_2	OR	d
1	-1	0	0	0	-1
2	-1	0	1	1	+1
3	-1	1	0	1	+1
4	-1	1	1	1	+1

- Then, we apply the training algorithm, pattern-by-pattern until no changes are possible:

$$\mathbf{w}(t+1) = \mathbf{w}(t) + \eta\left[d(t) - y(t)\right]\mathbf{x}(t)$$

$$d(t) = \begin{cases} +1 & \text{if } \mathbf{x}(t) \text{ belongs to } C_1 \\ -1 & \text{if } \mathbf{x}(t) \text{ belongs to } C_2 \end{cases}$$

Solution

- Currently weight vector is: **w**=[-1.5, 1, 1] (from initial values)
- **Epoch 1:**
  - **Process pattern 1, x=[-1,0,0], d=-1.**
  - y=ϕ(+1.5)=+1 --> misclassified, so change weights to:
  - [-1.5,1,1] + 0.5×(d-y)×[-1,0,0]= [-1.5,1,1] + -1× [-1,0,0]= [-0.5,1,1]
- Currently weight vector is: **w**=[-0.5, 1, 1] (1st weight update)
  - **Process pattern 2, x=[-1,0,1], d=+1.**
  - y=ϕ(+1.5)=+1 --> correctly classified, so nothing to change!
  - [-0.5,1,1] + 0.5×0×[-1,0,1]= same as before.
  - **Process pattern 3, x=[-1,1,0], d=+1.**
  - y=ϕ(+1.5)=+1 è correctly classified, so nothing to change!
  - [-0.5,1,1] + 0.5×0×[-1,1,0]= same as before.
  - **Process pattern 4, x=[-1,1,1], d=+1.**
  - y=ϕ(+2.5)=+1 --> correctly classified, so nothing to change!
  - [-0.5,1,1] + 0.5×0×[-1,1,1]= same as before.

Solution

- Currently weight vector is: **w**=[-0.5, 1, 1] (from previous 1st update)
- **Epoch 2:**
  - **Process pattern 1, x=[-1,0,0], d=-1.**
  - y=ϕ(+0.5)=+1 --> misclassified, so change weights to:
  - [-0.5,1,1] + 0.5 × (d-y) × [-1,0,0]= [-0.5,1,1] + -1 × [-1,0,0]= [0.5,1,1]
- Currently weight vector is: **w**=[+0.5, 1, 1] (2nd weight update)
  - **Process pattern 2, x=[-1,0,1], d=+1.**
  - y=ϕ(+0.5)=+1 --> correctly classified, so nothing to change!
  - [0.5,1,1] + 0.5 × 0 × [-1,0,1]= same as before.
  - **Process pattern 3, x=[-1,1,0], d=+1.**
  - y=ϕ(+0.5)=+1 --> correctly classified, so nothing to change!
  - [0.5,1,1] + 0.5 × 0 × [-1,1,0]= same as before.
  - **Process pattern 4, x=[-1,1,1], d=+1.**
  - y=ϕ(+1.5)=+1 --> correctly classified, so nothing to change!
  - [0.5,1,1] + 0.5 × 0 × [-1,1,1]= same as before.

Solution

- Currently weight vector is: **w**=[0.5, 1, 1] (from previous 2nd update)

Epoch 3:

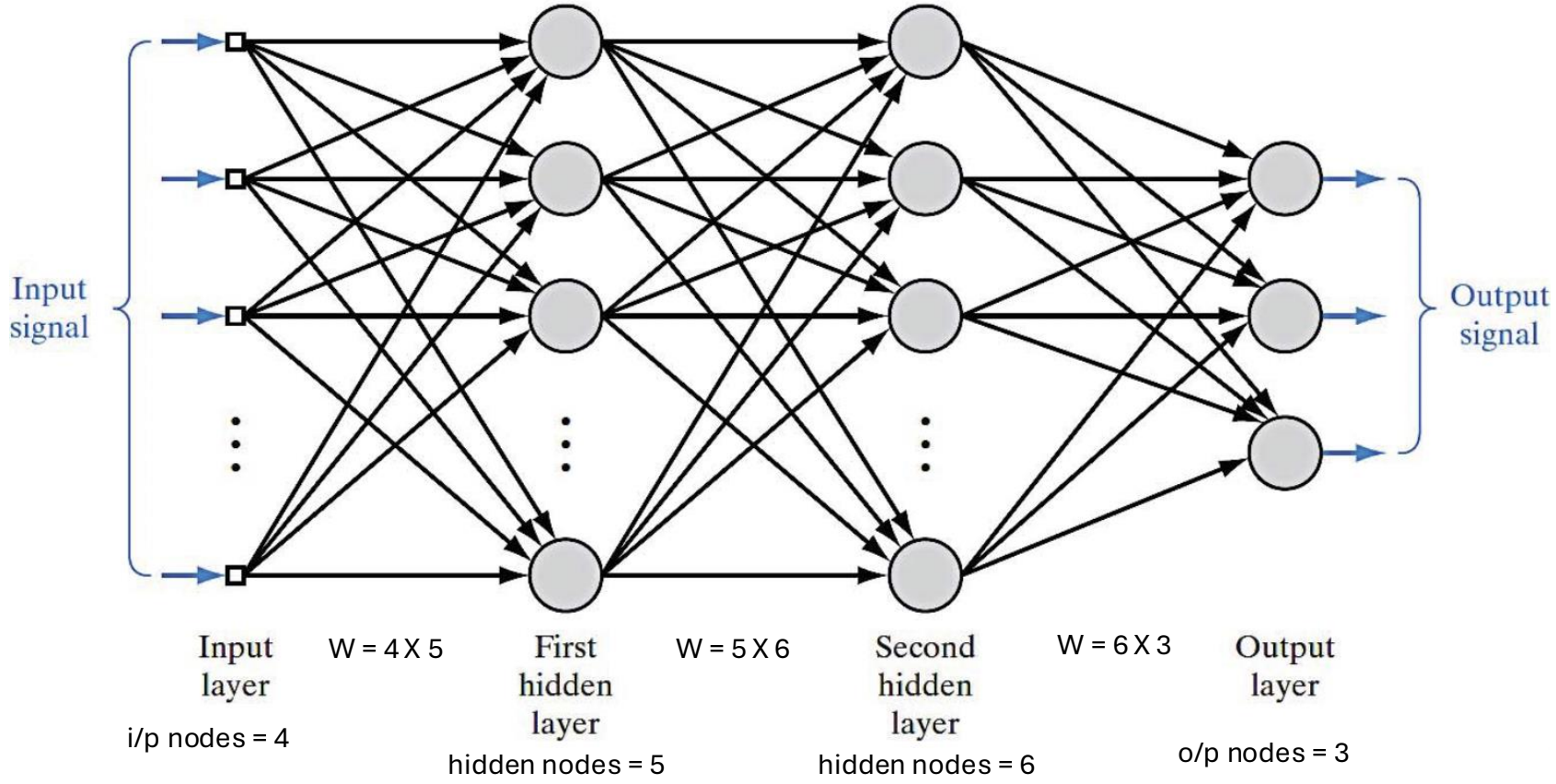
- Here we can try all patterns once more, in order to verify that all patters are correctly classified.
- We can do that, by trying one-by-one patterns as before, or using matrix calculations as:

$$X = \begin{bmatrix} -1 & 0 & 0 \\ -1 & 0 & 1 \\ -1 & 1 & 0 \\ -1 & 1 & 1 \end{bmatrix} \qquad w = \begin{bmatrix} 0.5 \\ 1 \\ 1 \end{bmatrix}$$

$$\text{Perceptron output} = \phi(\mathbf{Xw}) = \phi\left(\begin{bmatrix} -0.5 \\ 0.5 \\ 0.5 \\ 1.5 \end{bmatrix}\right) = \begin{bmatrix} -1 \\ +1 \\ +1 \\ +1 \end{bmatrix} = \text{Desired output}$$

Section 3: Artificial Neural Network

Multi-layer Perceptrons (MLPs)



Characteristics of MLPs

- Nonlinear activation functions which are smooth and differentiable (they cannot be the discontinuous Heaviside, for example). Examples include the logistic function which takes the ILF v_j (i.e., the weighted sum of all inputs to the j_{th} neuron and its bias) and transforms it to the output y_j:

$$y_j = \frac{1}{1 + \exp(-\alpha_j v_j)}$$

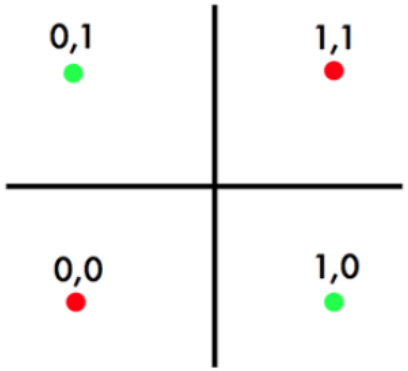
- Unlike SLPs, MLPs have one or more layers of neurons: the hidden layers. This allows the learning of more complex tasks, because each layer "generates" more complex features than its previous layer.
- They have high degrees of connectivity, since each node can receive input from any previous (only previous layer, to avoid feedback loops or cycles in the graphs).

The function of hidden layers

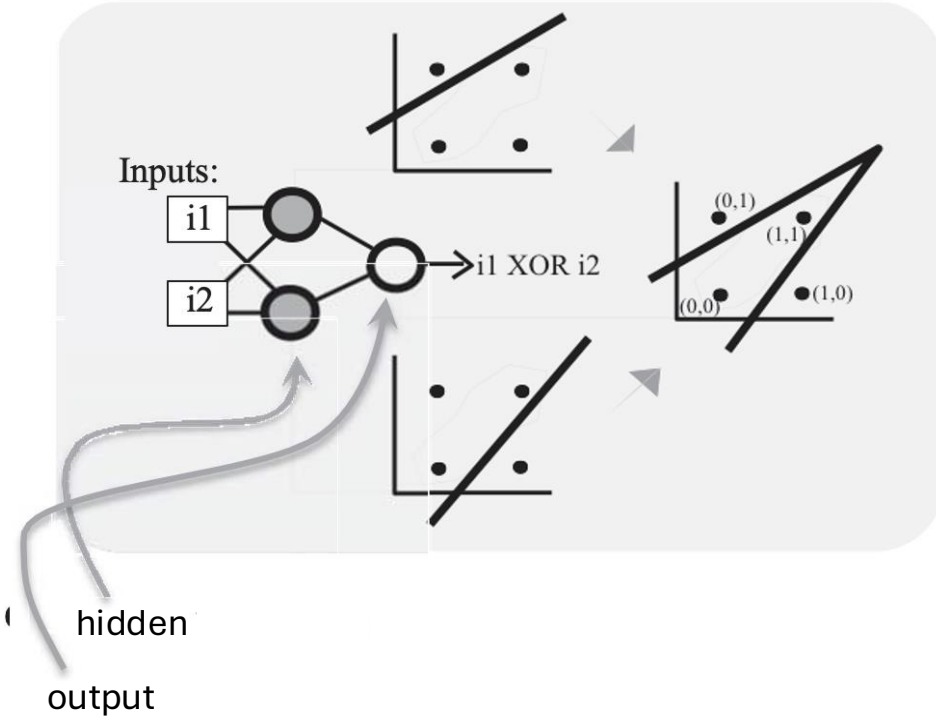
- The presence of hidden layers can allow complex representations. Consider the XOR problem for instance, which is not a linearly separable one:

• Each of the **hidden nodes** above, **finds a partial** solution to the problem

• Then, the **output** node **combines the two partial** solutions and makes the problem separable.

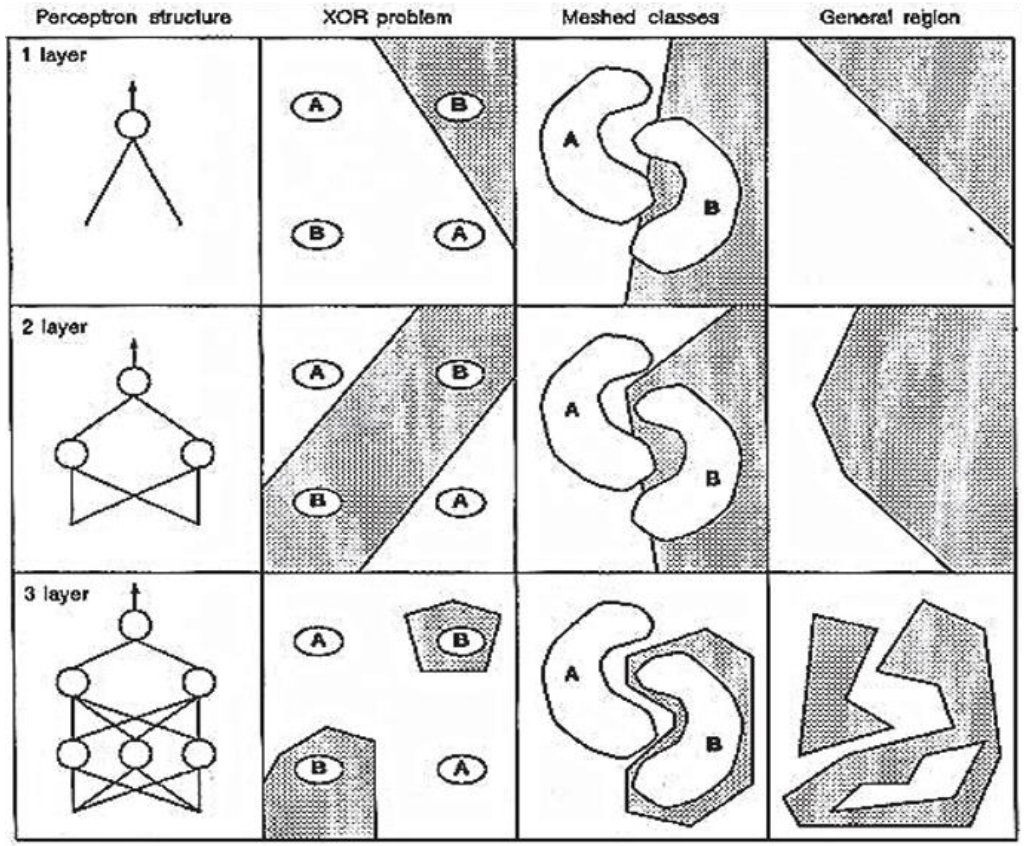


XOR



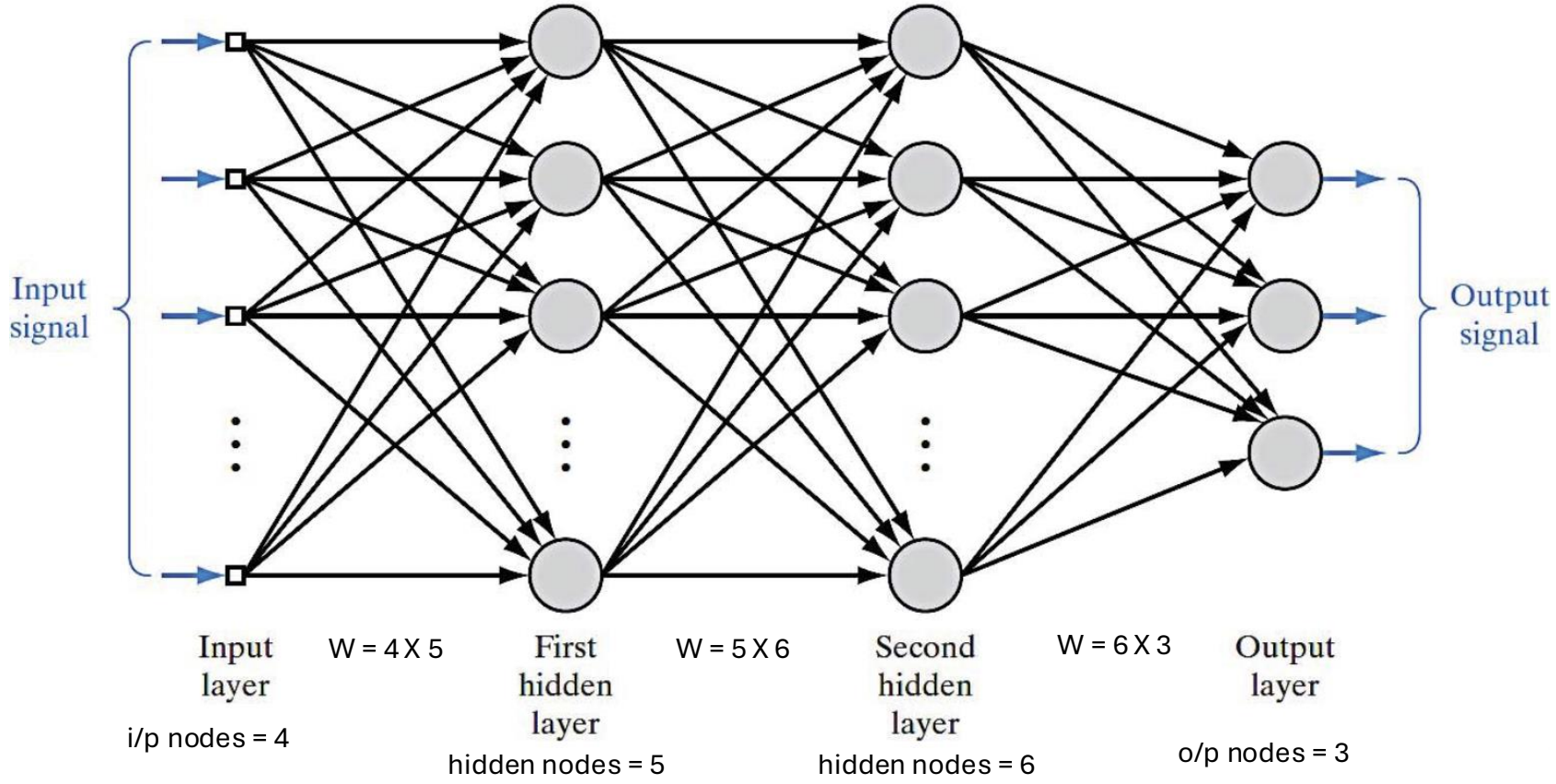
The function of hidden layers

- Any more layers? Of course! The **logic is the same**: more complex decision boundaries are progressively formed in the next layer:



Section 4: Backpropagation Algorithm

Multi-layer Perceptrons (MLPs)



The Back-Propagation (BP) Algorithm

- This is a very popular computationally efficient (but not unique) method for training MLPs. Although it is not optimal it discards the pessimism of some SLP pioneers about the uselessness of multi-layer learning!
- The algorithm is as follows:
  - We define the error signals at the j^{th} output neuron level with target output d_j and at the network level as:

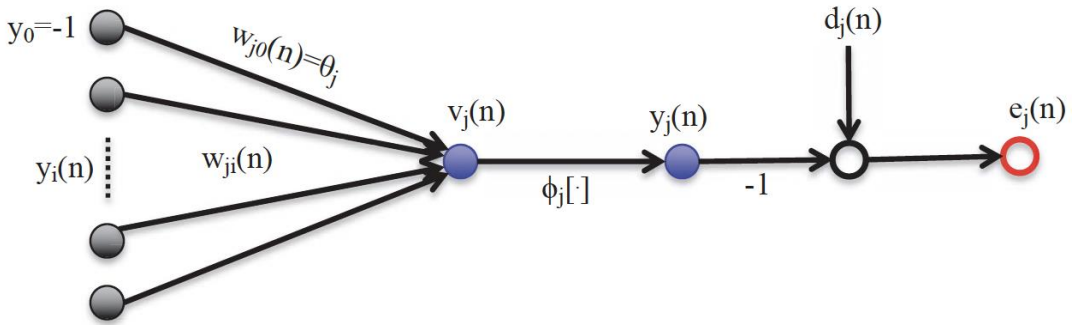
Output neuron error: $e_j(n) = d_j(n) - y_j(n)$

Output network error: $E(n) = \frac{1}{2} \sum_{j \in \text{Output}} e_j^2(n)$

Average net. output error: $E_{avg} = \frac{1}{N} \sum_{n=1}^{N} E(n)$ (for N data patterns)

where, Induced local field: $v_j(n) = \sum_{i=0}^{p} w_{ji}(n) y_i(n)$

Neuron output: $y_j(n) = \phi_j[v_j(n)]$



The Back-Propagation (BP) Algorithm

- We now wish to train the MLP using our available data patterns ($\mathbf{x}_i, \mathbf{d}_i$), i=1,…,N.
  - But the goal is always the sa $\Delta w_{ji}(n) \equiv w_{ji}(n+1) - w_{ji}(n)$
    - Adapt each synaptic weight:
  - First we calculate the derivative of the network error E(n) at time n with respect to the syna

$$\frac{\partial E(n)}{\partial w_{ji}(n)} = \underbrace{\frac{\partial E(n)}{\partial e_j(n)}}_{e_j(n)} \underbrace{\frac{\partial e_j(n)}{\partial y_j(n)}}_{-1} \underbrace{\frac{\partial y_j(n)}{\partial v_j(n)}}_{\phi_j'[v_j(n)]} \underbrace{\frac{\partial v_j(n)}{\partial w_{ji}(n)}}_{y_i(n)} = \overbrace{-e_j(n)\phi_j'[v_j(n)]}^{\delta_j(n)} y_i(n) =$$

$$= -\delta_j(n) y_i(n)$$

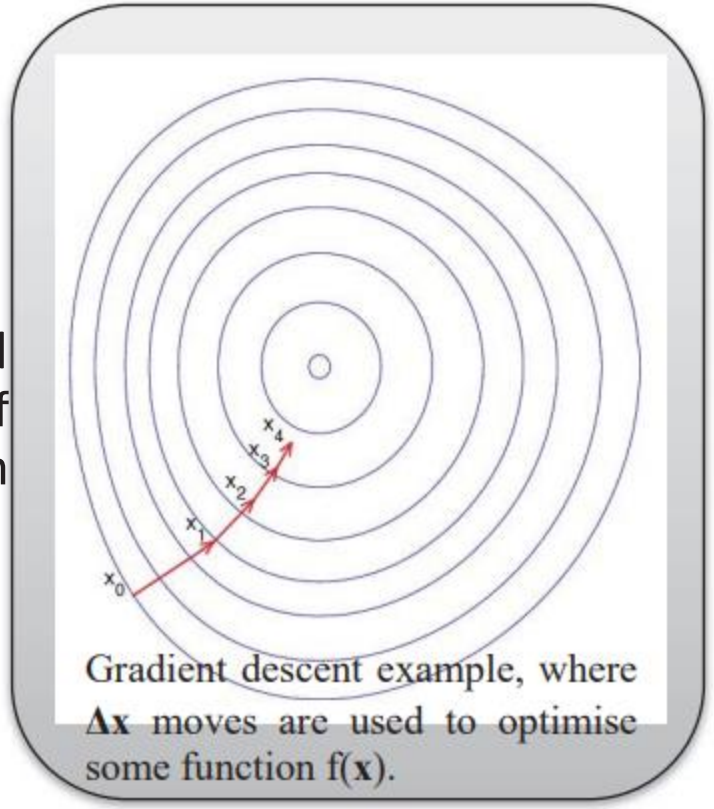
The Back-Propagation (BP) Algorithm

- So, given a learning rate η, a simple gradient descent optimisation, would move in the weight space towards the direction of:

$$\Delta w_{ji}(n) = -\eta \frac{\partial E(n)}{\partial w_{ji}(n)} = \eta \delta_j(n) y_i(n) \quad (1)$$

- This direction which minimises E(n), is based on the newly introduced local gradient of the j^{th} neuron, which is defined as the product of the error signal of that neuron and the derivative of its activation with respect to the ILF of the neuron:

$$\delta_j(n) \equiv e_j(n) \phi_j'[v_j(n)] \quad (A)$$



Gradient descent example, where **Δx** moves are used to optimise some function f(**x**).

The Back-Propagation (BP) Algorithm

- Thus, to calculate Δw_{ji} the error signal at the output of the j^{th} neuron is needed. But **not all** neurons have directly known output! This gives rise to two cases:
  - **j is an output neuron:** Here it is easy to calculate the weight change Δw_{ji}, since $e_j(n)=d_j(n)-y_j(n)$, and d_j is known by the training dataset.
  - **j is a hidden neuron:** Here the error is **not** directly known, **but** these neurons surely have a responsibility for the error observed in the output layer.
- But how can we penalise or reward the hidden neuron when their errors $e_j(n)$ are not directly available because their desired responses $d_j(n)$ are not? This is an instance of the ***Credit Assignment Problem***!
- This is solved by "back-propagating" the signal error from the output layer through the previous hidden layers, as follows:
  - Using the previous definitions, we can conceptually redefine the local gradient as:

$$\delta_j(n) = -\frac{\partial E(n)}{\partial y_j(n)}\frac{\partial y_j(n)}{\partial v_j(n)} = \boxed{-\frac{\partial E(n)}{\partial y_j(n)}\phi_j'[v_j(n)]}^{2} = e_j(n)\phi'[v_j(n)]$$

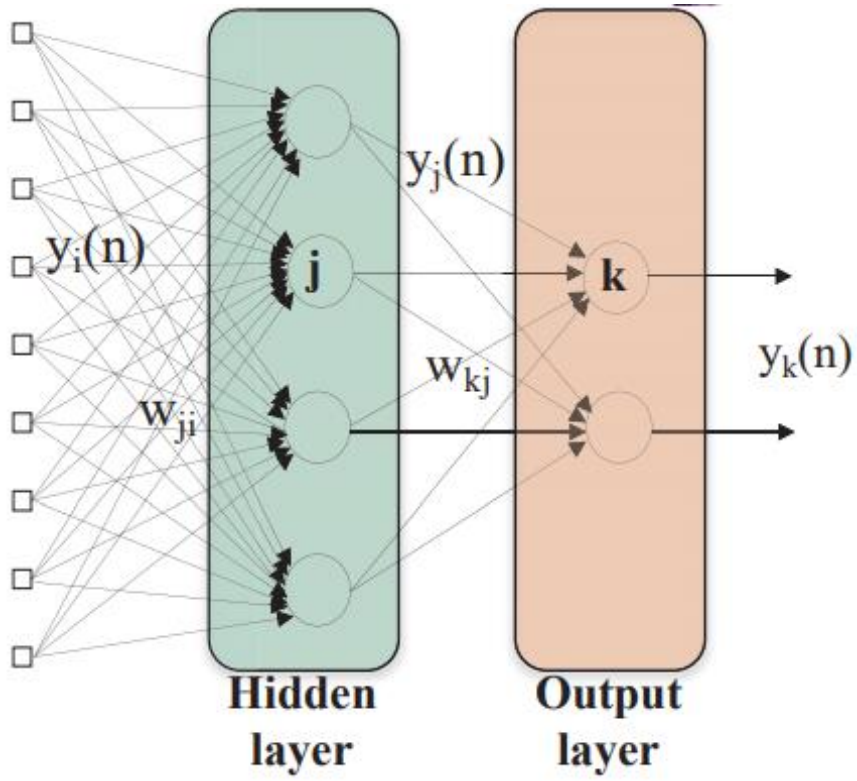
The Back-Propagation (BP) Algorithm

$$\frac{\partial E(n)}{\partial y_j(n)} = -\sum_k \underbrace{e_k(n)\phi'_k\left[v_k(n)\right]}_{\delta_k(n)} w_{kj}(n)$$

- Finally, inserting the above into Eq(2), we get the back-propagation formula for the local gradient in the j^{th} hidden neuron:

$$\delta_j(n) = \phi'_j\left[v_j(n)\right]\sum_k \delta_k(n) w_{kj}(n) \quad \text{B}$$

- Eq(B) allows us to apply the weight change delta-rule of Eq(1) to hidden neurons. It can be seen that the δ_j of a hidden node can be calculated in terms of the δ_k of the k^{th} neurons connected (i.e., receiving signals) from neuron j.
- This means that Eq(B) is valid for j being a neuron in any hidden layer, whilst neuron k can be a neuron from the next hidden **or** the output layer.
- Overall, Eq(A) is used for the output layer, and Eq(B) for all the hidden ones.



The Back-Propagation (BP) Algorithm: Full Summary

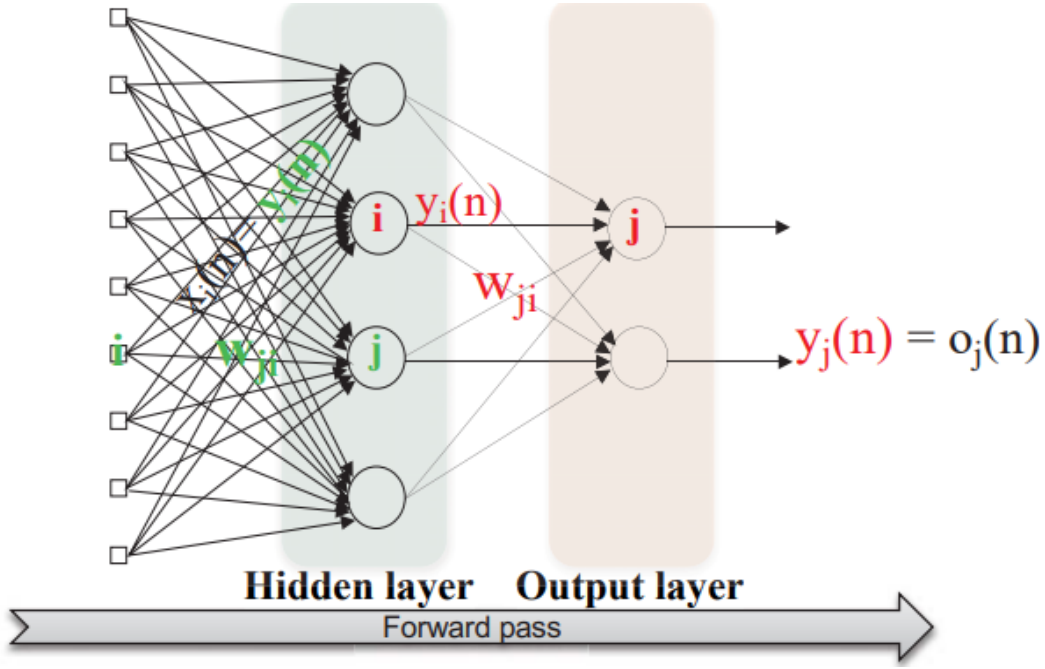
- The BP algorithm for training MLPs consists of two passes of computation, which are alternatively applied.

Forward Pass

- All weights remain fixed.
- We start from the first hidden layer and for all its neurons we calculate their outputs and ILFs as:

$$y_j(n) = \phi_j[v_j(n)], \quad v_j(n) = \sum_{i=0}^{p} w_{ij}(n) y_i(n)$$

- If the above is applied in the first hidden layer all y_i (n) simple represent the network input signal $X_i(n)$
- We repeatedly move to the next layer, until we reach the output layer where $y_j(n)$ is the j^{th} network output, typically also written as $o_j(n)$.
- If the above is applied to the 2^{nd},3^{rd}, etc. Hidden layer or the output layer, $y_i(n)$ is output signal of the i^{th} node in the previous hidden layer.



The Back-Propagation (BP) Algorithm: Full Summary

Backward Pass

- This pass starts from the last (the output layer) and works backwards a layer-by-layer fashion, until it reaches the first hidden layer
- All input signals to nodes are clamped during this pass
- For each j^{th} node of the current layer, it calculates the local gradient using:

  - Eq(A), i.e. $\delta_j(n) = e_j(n)\phi'_j\left[v_j(n)\right]$

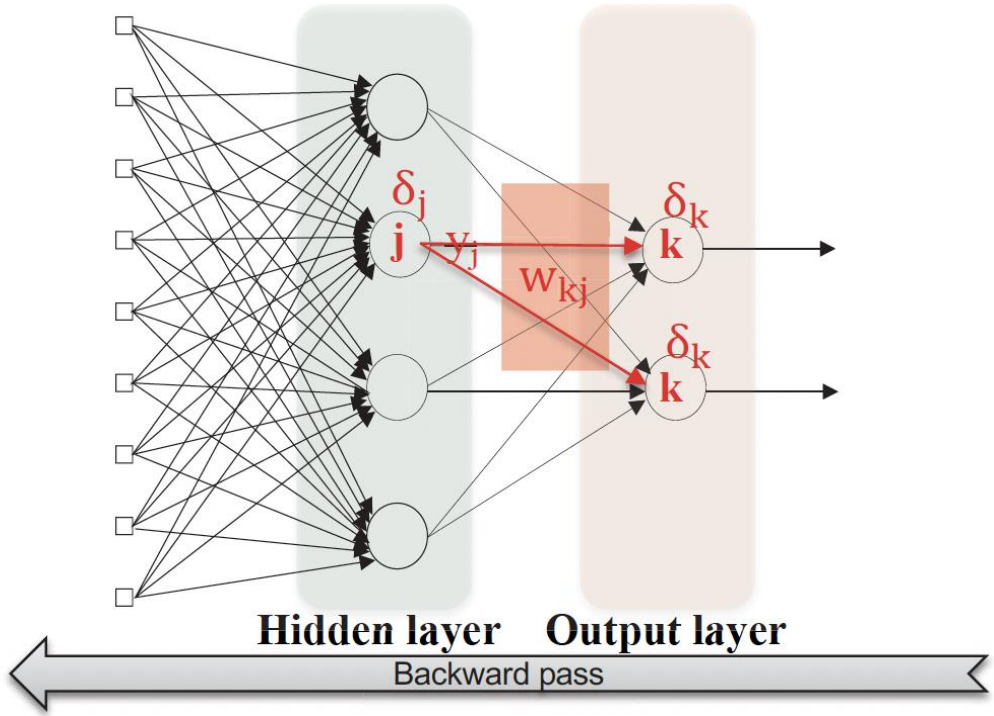
    if the node is in the output layer, or

  - Eq(B), i.e. $\delta_j(n) = \phi'_j\left[v_j(n)\right]\sum_k \delta_k(n) w_{kj}(n)$

    if the node is in any hidden layer.

- All the local gradients are then used in Eq(1) to calculate and adapt the synaptic weights so that to reduce the network error:

$$\Delta w_{ji}(n) = -\eta\frac{\partial E(n)}{\partial w_{ji}(n)} = \eta\delta_j(n)y_i(n)$$

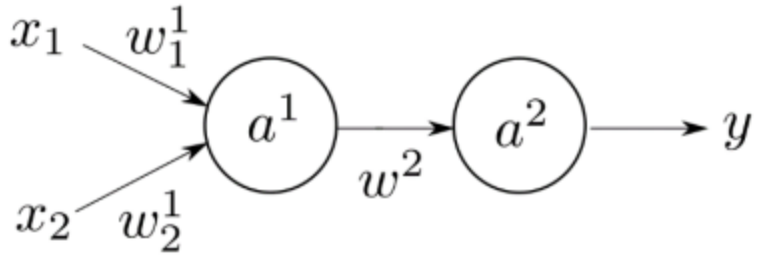


The Back-Propagation (BP) Algorithm: Example

Let us consider a two layer neural network with the diagram.

The neural network has two inputs $\mathbf{x_1}$ and $\mathbf{x_2}$ and one output y.

The equations that represent this neural network are



$$a^1 = \phi_1(w_1^1 x_1 + w_2^1 x_2)$$
$$a^2 = \phi_2(w^2\, a^1)$$
$$y = a^2$$

Where the first activation function is linear: $\phi_1(x) = \alpha x,$ and the second activation function is a sigmoid $\phi_2(x) = \frac{1}{1+e^{-x}}$.

Let us consider the weights are $w_1^1 = 1, w_2^1 = 0.1,$ and $w^2 = 0.1,$ and the parameter α = 0.5.

1) For each neuron calculate its induced local field and output for the following input value $x_1 = 1, x_2 = 2.$

2) If the desired output to the previous input is d = 1, use backpropagation algorithm to compute the gradient of the error with respect to the weights.

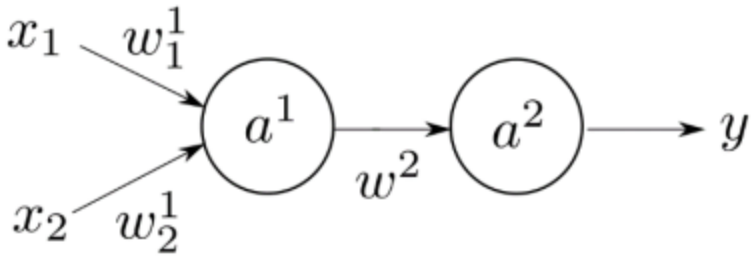
3) If the learning rate is 0.1, calculate the updated weights.

4) Calculate the output to the input value $x_1 = 1, x_2 = 2$ with the updated weights.

The Back-Propagation (BP) Algorithm: Example

1) Induced local field and output neuron 1

$$v_1 = w_1^1 x_1 + w_2^1 x_2 = 1.2$$
$$a^1 = 0.60$$



Induced local field and output neuron 2

$$v_2 = w^2 a^1 = 0.06$$
$$a^2 = 0.5150$$

The Back-Propagation (BP) Algorithm: Example

2) Backpropagation. We first calculate the local gradients δ_1 and δ_2

$$\delta_2 = e_2 \phi_2'(v_2)$$
$$e_2 = d - y = 1 - 0.5150$$
$$\phi_2'(v_2) = \frac{e^{-v_2}}{(1+e^{-v_2})^2}$$

Then,

$$\delta_2 = 0.1211$$

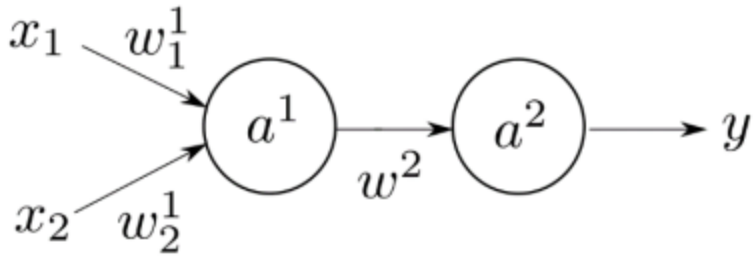
To calculate $\delta_1 = \phi_1'(v_1)\delta_2 w^2$

Then, $\phi_1'(v_1) = \alpha$

$$\delta_1 = 0.5 \times 0.1211 \times 0.1 = 0.0061$$

The gradients are then

$$\frac{\partial E}{\partial w^2} = -\delta_2 a^1 = -0.0727$$
$$\frac{\partial E}{\partial w_1^1} = -\delta_1 x_1 = -0.0061$$
$$\frac{\partial E}{\partial w_2^1} = -\delta_1 x_2 = -0.0121$$



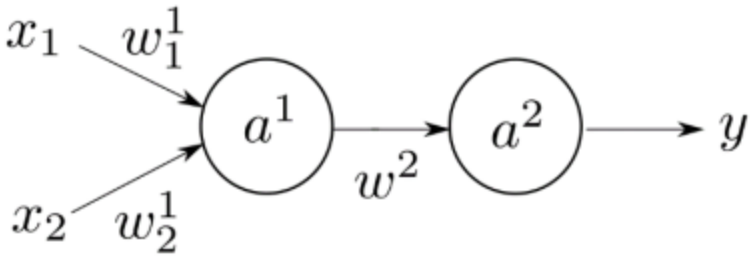
The Back-Propagation (BP) Algorithm: Example

3) Updated weights

$$w_1^{1,+} = w_1^1 - \eta \frac{\partial E}{\partial w_1^1} = 1.0006$$

$$w_2^{1,+} = w_2^1 - \eta \frac{\partial E}{\partial w_2^1} = 0.1012$$

$$w^{2,+} = w^2 - \eta \frac{\partial E}{\partial w^2} = 0.1073$$



4) The new output is y= 0.5161. In the previous step the output was 0.5150 and the desired output is 1. Therefore, after backpropagation, the weights have been modified to lower the error.

Neural Networks: A Quick Overview

- A neural network models a function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$
- The output is computed through a series of matrix multiplications followed by non-linear activations

$$y = \sigma(Wx + b)$$

Where:

- x is the input vector
- W is the weight matrix
- b is the bias vector
- σ is a non-linear activation function, such as ReLU or sigmoid

Thank You!!!!!!